

Facultad de Ciencias UNAM

Tarea 2: Tiempos de ejecución y orden asintótico

Profesora: Virginia Teodosio Procopio
Ayudantes: Leonardo Gallo Guerrero
José Manuel Madrigal Ramírez

Entrega: 12 de Septiembre de 2026

Instrucciones

Resuelve los siguientes ejercicios, puede ser a mano o en latex y escribe tus respuestas en un archivo `.pdf` o `.md` cuyo nombre debe ser:

`t02.pdf`

En seguida crea una carpeta dentro de tu repositorio privado de Git con el nombre: `\tareas`, donde guardaremos el resto de las tareas del curso. Dentro de esa carpeta añade un archivo llamado `indice.md`, en el que guardes una lista a las tareas correspondientes con una estructura similar a la siguiente:

ÍNDICE DE TAREAS

```
2.Tiempos de ejecución y orden asintótico
3.
.
.
.
9. Colecciones
```

El código `.md` para conseguir esto obedece una sintaxis como la siguiente:

```
# ÍNDICE DE TAREAS
2. [Axiomatización](t02.pdf)
3. ...
.
.
.
9 [Colecciones](t09.pdf)
```

En caso de que realices la tarea **en pareja**, ambos miembros del equipo deben subir el archivo a sus respectivos repositorios y escribir el nombre y número de cuenta de ambxs en el archivo `.pdf` de las respuestas a los ejercicios.

De este modo la carpeta que añadas a tu repositorio de Git debería tener la siguiente estructura:

```
Tareas/  
├── indice.md  
└── t02.pdf
```

Finalmente, añade a tu archivo `README.md` en la sección de **Organización**, un enlace al índice de tus tareas . Recuerda que, como indicó Leo en nuestro grupo del curso, tu archivo `README.md` debe contener las siguientes secciones:

- Título y Descripción
- Tecnologías
- Prerrequisitos
- **Organización** (Aquí debes añadir el enlace al índice de tus tareas).
- Instalación
- Uso
- Pruebas (Testing)
- Contribución
- Licencia y Créditos

Ejercicio 1: Conceptos fundamentales (3 ptos.)

1. Explica qué se entiende por **complejidad de entrada**. Menciona al menos tres características de los datos de entrada que podrían afectar el desempeño de un algoritmo.
2. ¿Qué es una **función de complejidad**? Explica qué relación representa entre la entrada de un algoritmo y los recursos que utiliza.
3. Supongamos que dos estudiantes ejecutan el mismo algoritmo en computadoras diferentes y obtienen tiempos distintos. Explica por qué esto no necesariamente significa que alguno de los dos los haya medido incorrectamente.
4. En clase definimos, dentro de nuestro modelo teórico, el tiempo de ejecución de un algoritmo como la cantidad de **operaciones básicas** que realiza. Explica por qué esta definición puede ser útil para estudiar y comparar algoritmos en vez de partir de hacer experimentos y tratar de definir funciones de complejidad a partir de graficar los resultados obtenidos.

5. Menciona al menos dos factores del entorno de ejecución que podrían producir variaciones en los tiempos obtenidos experimentalmente cuando programamos un algoritmo y medimos el tiempo que le toma terminar para diversas entradas.
6. Explica con tus propias palabras qué significa afirmar que un algoritmo tiene orden

$$O(g(n)).$$

No es necesario dar una definición formal mediante constantes; en esta pregunta nos interesa principalmente la interpretación del concepto.

Ejercicio 2: Cálculo de tiempo y orden asintótico (5 pts.)

Al final de la clase del pasado jueves 3 de septiembre, en la que abordamos ejemplos de conteo de operaciones básicas sobre algoritmos sencillos y que se encuentra disponible en el sitio de la materia, enunciamos las siete operaciones básicas que utilizaremos al contar el tiempo de ejecución de un algoritmo.

1. Enuncia las siete operaciones:
2. Utilizándolas como marco conceptual, para cada uno de los siguientes algoritmos:
 - a Escribe el número de operaciones básicas que se ejecutan en el peor de los casos.
 - b Deduce su función de complejidad.
 - c Exhibe el orden asintótico al que pertenece y justifica tu respuesta.

Algoritmo 1 Problema1(n)

Precondiciones: n es un Entero positivo

```
1:  $suma \leftarrow 0$ 
2: for  $i \in \{0, \dots, n-1\}$  do
3:    $suma \leftarrow suma + i$ 
4: end for
5: return  $suma$ 
```

Algoritmo 2 Problema2(n)

Precondiciones: n es un Entero positivo

```
1:  $suma \leftarrow 0$ 
2: for  $i \in \{0, \dots, n-1\}$  do
3:   for  $j \in \{0, \dots, i\}$  do
4:      $suma \leftarrow suma + 1$ 
5:   end for
6: end for
7: return  $suma$ 
```

Algoritmo 3 Problema3(n)

Precondiciones: n es una potencia entera de 2 y $n > 0$

```
1:  $i \leftarrow 1$ 
2:  $contador \leftarrow 0$ 
3: while  $i < n$  do
4:    $i \leftarrow 2i$ 
5:    $contador \leftarrow contador + 1$ 
6: end while
7: return  $contador$ 
```

Algoritmo 4 Problema4(n)

Precondiciones: n es un Entero positivo

```
1:  $x \leftarrow 0$ 
2: for  $i \in \{0, \dots, n-1\}$  do
3:    $x \leftarrow x + 1$ 
4: end for
5: for  $j \in \{0, \dots, n-1\}$  do
6:    $x \leftarrow x + 1$ 
7: end for
8: return  $x$ 
```

Ejercicio 3: Comparación de algoritmos (2ptos.)

Supongamos que tenemos tres algoritmos para resolver el mismo problema y sus funciones de complejidad son:

$$T_A(n) = 20n + 50,$$

$$T_B(n) = 3n^2 + 4,$$

$$T_C(n) = 2n^3 + 7n.$$

- a) Determina el orden asintótico de cada algoritmo.
- b) ¿Cuál de los tres algoritmos parece ser más eficiente para entradas pequeñas?
- c) ¿Cuál esperamos que sea más eficiente cuando n sea muy grande?
- d) Explica por qué el orden asintótico es especialmente útil cuando queremos estudiar el comportamiento de los algoritmos para entradas grandes.



**PAUSA
PARA
TOMAR
AGUA**